

Arsitektur Modular OmniBit

Spesifikasi Implementasi Bit Packing Multi-Kapasitas (16, 32, 64, 128-bit) untuk Protokol Modbus

Dokumen ini menyediakan arsitektur kode bersih berorientasi masa depan yang memisahkan tanggung jawab (Separation of Concerns) untuk konversi data desimal Modbus menjadi bitmask status sensor/aktuator. Pemisahan kode (split-code) dilakukan agar modul ini modular, mudah diuji (testable), serta mendukung ekspansi register masif hingga 128-bit menggunakan fitur modern JavaScript BigInt.

Struktur File Project

Project dipecah menjadi tiga file utama untuk mempermudah maintenance:

Nama File	Tanggung Jawab	Deskripsi Singkat
constants.js	Konfigurasi & Konstanta	Menyimpan konfigurasi kapasitas bit (16, 32, 64, 128) beserta batasan matematis maksimumnya.
converter.js	Engine Core Logic	Berisi algoritma utama encoder dan decoder bit masking menggunakan kombinasi manipulasi bitwise klasik dan modern BigInt.
index.js	Aplikasi & Simulasi	Titik masuk (entry point) utama untuk mengeksekusi use-case, testing data register, serta integrasi input-output.

1. File Spesifikasi Konstanta

File ini bertugas mengunci definisi bit width agar terhindar dari magic numbers di dalam kode utama.

```
constants.js

/**
 * OmniBit - Data Width Configuration Constants
 * Mendefinisikan kapasitas bit dan nama representasinya dalam dunia register PLC/Modbus.
 */
export const BIT_WIDTHS = {
  WORD_16: 16,      // 1 Modbus Register (UINT16 / INT16)
  DWORD_32: 32,     // 2 Modbus Registers (UINT32)
  QWORD_64: 64,     // 4 Modbus Registers (UINT64)
  OWORD_128: 128    // 8 Modbus Registers (Octuple Word / UUID / Encryption Block)
};

// Batasan nilai aman desimal untuk masing-masing tipe data bit
export const BIT_LIMITS = {
  16: 65535n,
  32: 4294967295n,
  64: 18446744073709551615n,
  128: 340282366920938463463374607431768211455n
};
```

2. File Engine Core Converter

Logika inti penanganan bitwise. Operasi bitwise murni dipaksa menggunakan format BigInt (dengan akhiran n) untuk mencegah bug pemotongan 32-bit bawaan JavaScript.

converter.js

```
import { BIT_WIDTHS, BIT_LIMITS } from './constants.js';

/**
 * Mengonversi nilai desimal (BigInt/String/Number) menjadi array status boolean (Bit ke Sensor).
 * @param {string|number|bigint} decimalValue - Angka desimal yang didapat dari Modbus register.
 * @param {number} bitWidth - Ukuran bit (16, 32, 64, atau 128).
 * @returns {boolean[]} Array status true/false berurutan dari Bit 0 (indeks 0).
 */
export function decimalToBits(decimalValue, bitWidth = BIT_WIDTHS.WORD_16) {
  // Validasi apakah bitWidth didukung
  if (!Object.values(BIT_WIDTHS).includes(bitWidth)) {
    throw new Error(`Kapasitas bit ${bitWidth} tidak didukung oleh OmniBit.`);
  }

  // Ubah ke BigInt dengan aman
  let bigDecimal = BigInt(decimalValue);

  // Proteksi overflow nilai register
  if (bigDecimal < 0n || bigDecimal > BIT_LIMITS[bitWidth]) {
    throw new RangeError(`Nilai desimal di luar batas aman untuk kapasitas $
{bitWidth}-bit.`);
  }

  const sensorStatus = [];

  // Lakukan bitmasking menggunakan BigInt shift operator
  for (let i = 0; i < bitWidth; i++) {
    const bitMask = 1n << BigInt(i);
    const isActive = (bigDecimal & bitMask) !== 0n;
    sensorStatus.push(isActive);
  }

  return sensorStatus;
}

/**
 * Mengonversi array status boolean kembali menjadi satu nilai desimal BigInt (Sensor ke Bit).
 * @param {boolean[]} bitsArray - Array penanda aktifnya sensor/aktuator.
 * @param {number} bitWidth - Batasan ukuran bit target.
 * @returns {bigint} Nilai desimal siap kirim ke Modbus Register.
 */
export function bitsToDecimal(bitsArray, bitWidth = BIT_WIDTHS.WORD_16) {
  let bigDecimal = 0n;

  // Batasi proses looping hanya sejauh kapasitas bitWidth yang ditentukan
  const length = Math.min(bitsArray.length, bitWidth);

  for (let i = 0; i < length; i++) {
```

```
        if (bitsArray[i] === true) {  
            bigDecimal |= (1n << BigInt(i));  
        }  
    }  
  
    return bigDecimal;  
}
```

3. File Entry Point & Use-Case Simulasi

File ini mendemonstrasikan bagaimana modul dijalankan untuk melakukan decode status sensor di berbagai arsitektur Modbus data word.

index.js

```
import { BIT_WIDTHS } from './constants.js';
import { decimalToBits, bitsToDecimal } from './converter.js';

console.log("=== OMNIBIT BITPACKING ENGINE RUNNING ===");

// =====
// KASUS 1: 16-Bit Modbus (1 Word Standard)
// =====
console.log("
[KASUS 1] Modbus 16-Bit (1 Word)");
const nilaiModbus16 = 13; // Biner: 1101 (Sensor 0, 2, 3 aktif)
const statusSensor16 = decimalToBits(nilaiModbus16, BIT_WIDTHS.WORD_16);

console.log(`Desimal Masuk: ${nilaiModbus16}`);
console.log("Status 4 Sensor Pertama:", statusSensor16.slice(0, 4));
// Output: [true, false, true, true]

// =====
// KASUS 2: 64-Bit Modbus (4 Registers Block)
// =====
console.log("
[KASUS 2] Modbus 64-Bit (4 Registers)");
// Contoh nilai masif di mana bit ke-60 aktif (Skenario sensor di alamat tinggi)
const nilaiMasifDesimal = "1152921504606846976"; // 1n << 60n
const statusSensor64 = decimalToBits(nilaiMasifDesimal, BIT_WIDTHS.QWORD_64);

console.log(`Desimal Masuk: ${nilaiMasifDesimal}`);
console.log(`Apakah Sensor ke-60 Aktif? ${statusSensor64[60]}`); // Output: true
console.log(`Apakah Sensor ke-61 Aktif? ${statusSensor64[61]}`); // Output: false

// =====
// KASUS 3: Re-encoding (Bit Ke Desimal)
// =====
console.log("
[KASUS 3] Re-encoding Array ke Desimal");
const dataSimulasiSensor = new Array(128).fill(false);
dataSimulasiSensor[0] = true; // Sensor 0 Aktif
dataSimulasiSensor[127] = true; // Sensor 127 Aktif (Ujung Bit masif 128-bit)

const hasilDesimal128 = bitsToDecimal(dataSimulasiSensor, BIT_WIDTHS.OWORD_128);
console.log("Hasil Desimal Pack 128-bit:", hasilDesimal128.toString());
```

Rekomendasi Integrasi Modbus: Saat membaca data 32, 64, atau 128 bit dari library Modbus (seperti `modbus-serial` atau `node-modbus`), Anda akan menerima data dalam bentuk array buffer/register integers (misal: 4 array berisikan data 16-bit). Gabungkan register tersebut terlebih dahulu menggunakan operasi shift biner sebelum memasukkannya ke dalam engine **OmniBit** ini.